

Big Data Processing II

IA Generativa aplicada al deporte

Prompt Engineering & Post-training

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Marzo 2026



Universidad
Rey Juan Carlos



Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>

Mapa del bloque



Sesión 1

Control del comportamiento

Prompt Engineering + Post-training



Sesión 2

Control de la información

Retrieval-Augmented Generation (RAG)



Sesión 3

Control del flujo y la acción

Agentes, Tools y Arquitecturas Cognitivas

Contexto



- Entender qué es un LLM desde el punto de vista computacional.
- Comprender su papel en la Ciencia de Datos Deportivos.
- Aprender a controlar su comportamiento mediante Prompt Engineering.
- Identificar límites del prompting.



- Modelo probabilístico de generación de texto.
- Aprende patrones lingüísticos.
- Predice la siguiente palabra dada una secuencia.
- Simula coherencia, no conocimiento real.

Arquitectura conceptual de un modelo de lenguaje

Esquema conceptual original de la edición abierta



- **No es una base de datos.**
- **No tiene acceso** a sensores o APIs.
- **No realiza cálculos fiables** por sí mismo.
- **No distingue verdad de plausibilidad.**

Datos deportivos y modelos generativos

Esquema conceptual original de la edición abierta



- Generar narrativas.
- Estandarizar informes.
- Traducir estadísticas a lenguaje humano.
- Interfaz conversacional.

Ciclo de diseño y evaluación

Esquema conceptual original de la edición abierta

Prompt Engineering



Un LLM no se “usa”, se **configura mediante lenguaje**.

- El prompt es el mecanismo con el que definimos:
 - Qué tarea debe realizar.
 - Qué información puede usar.
 - Qué formato debe producir.
- En un sistema de datos, el prompt actúa como:
 - Capa de control sobre el comportamiento del modelo.
 - Sustituto de una interfaz de programación tradicional.
- El modelo no cambia internamente → solo cambia su comportamiento en esa interacción.

Idea clave: Prompt Engineering = programación en lenguaje natural.



Un error común es tratar al LLM como si fuera un buscador o una persona.

- **Pregunta informal:**

“Resume el partido de ayer.”

- Ambigua.
- Sin contexto.
- Sin criterio de calidad.

- **Especificación de tarea:**

*“Genera un resumen técnico de máximo 5 líneas usando SOLO las métricas dadas.
Estructura: contexto, acciones clave, impacto táctico.”*

Diferencia fundamental: Las preguntas producen texto. Las especificaciones producen resultados utilizables en un sistema.



Un prompt profesional no es una frase, es una estructura:

- **Rol (Role)** Define el perfil desde el que debe responder el modelo. Ej.: “Eres analista de rendimiento en un club profesional”.
- **Tarea (Task)** Especifica exactamente qué debe hacer. Ej.: “Genera un informe técnico del jugador X”.
- **Contexto (Context)** Información disponible para realizar la tarea. Ej.: estadísticas, partido, periodo, posición del jugador.
- **Restricciones (Constraints)** Limita el comportamiento del modelo. Ej.: “No inventes datos”, “Usa solo las métricas dadas”.
- **Formato de salida (Output Format)** Cómo debe entregarse la respuesta. Ej.: JSON, tabla, bullets, informe estructurado.

Idea clave: un buen prompt reduce grados de libertad del modelo.



Prompt: *“Analiza el rendimiento del jugador.”*

- **Ambigüedad de objetivo:** ¿rendimiento ofensivo, defensivo, físico, táctico?
- **Sin contexto:** no hay periodo (partido, mes, temporada) ni rival ni rol.
- **Sin datos:** el modelo tenderá a **inventar** métricas o asumirlas.
- **Sin criterio:** no defines qué significa “bueno/malo” (percentiles, umbrales).
- **Sin formato:** imposible integrar la salida en un pipeline (ETL/dashboard).

Diagnóstico Big Data: prompt mal definido = salida no reutilizable + riesgo alto de alucinación.



Prompt (plantilla):

- **Rol:** “Eres analista de rendimiento en un club profesional”.
- **Tarea:** “Genera un informe técnico del jugador X”.
- **Contexto:** “Usa SOLO las métricas proporcionadas (tabla adjunta)”.
- **Restricciones:** “No inventes datos; si falta una métrica, marca null”.
- **Formato:** “Devuelve JSON con: resumen, puntos fuertes, puntos débiles, alertas”.

Efecto: reduces grados de libertad y conviertes la interacción en una “especificación”.



Entrada (ejemplo): métricas por 90 minutos (últimos 5 partidos)

- minutes=432, goals=2, assists=1, xG=1.6, xA=0.9
- shots_p90=2.8, key_passes_p90=1.7, passes_acc=84%
- pressures_p90=14.2, tackles_p90=1.1, interceptions_p90=0.8
- distance_km=10.9, sprints=21, high_intensity_runs=52

Nota: esto es intencionalmente “poco” para forzar que el prompt gestione faltantes sin inventar.



Objetivo: que la salida sea consumible por procesos automáticos (API/ETL/dashboard)

```
{
  "player": "X",
  "period": "last_5_matches",
  "summary": "... (3-5 lineas)",
  "strengths": ["...", "..."],
  "weaknesses": ["...", "..."],
  "alerts": [{"type": "fatigue_risk", "evidence": "..."}],
  "metrics_used": ["minutes", "xG", "xA", "pressures_p90"],
  "missing_metrics": ["duels_won_pct", "progressive_passes_p90"]
}
```

Clave: el modelo no “se luce”; produce un contrato de datos.



- **Trazabilidad:** puedes auditar qué campos rellena y cuáles faltan.
- **Evaluación automática:** validación de esquema (JSON schema), checks.
- **Integración:** dashboard/BI, storage, pipelines batch/stream.
- **Reproducibilidad:** mismo prompt + mismos datos → salida comparable.
- **Mitigación de alucinación:** forzar null/unknown evita inventar.

Regla de oro: si lo vas a usar en un sistema, **no pidas texto libre**.



- Un prompt operativo funciona como un **contrato** entre:
 - **Entrada** (datos disponibles) y **Salida** (estructura/criterios).
 - **Responsabilidades** del modelo (qué puede inferir vs qué no).
- En un proyecto real debes:
 - **Versionar** prompts (v1, v1.1...) igual que código.
 - Registrar: modelo, temperatura, prompt, y cambios de esquema.
 - Mantener tests con ejemplos (golden set) para evitar regresiones.

Puente a técnicas: para consistencia, entra Zero/One/Few-shot.



- Un prompt largo no es “mejor”: puede introducir contradicciones o ruido.
- Buen prompt = **instrucciones cortas + estructura + ejemplos mínimos**.
- Coste típico en sistemas: tokens (entrada + salida) $\Rightarrow$ dinero + latencia.
- Estrategia práctica:
 - Plantillas reutilizables.
 - Contexto **solo** necesario.
 - Ejemplos (few-shot) compactos.
 - Formato fijo para minimizar longitud de salida.

Mensaje para el máster: Prompt engineering también es **optimización de sistema**.

Técnicas de prompting



- El modelo resuelve la tarea solo con instrucciones.
- Se apoya en su conocimiento general aprendido en pretraining.
- Ventajas:
 - Bajo coste en tokens.
 - Rápido de diseñar.
 - Flexible ante tareas variadas.
- Limitaciones:
 - Menor consistencia de formato.
 - Más variabilidad en la respuesta.



Tarea: Clasificar tipo de evento táctico.

Prompt:

“Clasifica el siguiente evento como: transición ofensiva, ataque posicional o contraataque: ‘Recuperación en campo propio, pase largo al espacio, finalización en 8 segundos.’”

Observación: funciona, pero puede variar la explicación y formato.



- Se añade un ejemplo de entrada–salida correcta.
- El modelo imita patrón, estructura y nivel de detalle.
- Útil para:
 - Informes de scouting.
 - Clasificaciones consistentes.
 - Resúmenes con plantilla fija.



Ejemplo dado al modelo:

Entrada: métricas de un jugador.

Salida: informe estructurado con fortalezas, debilidades y alerta física.

Luego se pide lo mismo para otro jugador.

Efecto: se estandariza la estructura sin entrenar el modelo.



- Varios ejemplos refuerzan el patrón deseado.
- Reduce ambigüedad semántica.
- Mejora estabilidad del formato.
- Especialmente útil cuando:
 - hay terminología técnica
 - la tarea es subjetiva (evaluaciones)

Aprendizaje mediante ejemplos en contexto

Esquema conceptual original de la edición abierta



Comparación zero-shot, one-shot y few-shot

Esquema conceptual original de la edición abierta



- Obliga al modelo a generar pasos intermedios.
- Aumenta probabilidad de razonamientos coherentes.
- Reduce errores en tareas lógicas y causales.
- No es razonamiento real, es secuencialización probabilística.

Descomposición explícita de una tarea

Esquema conceptual original de la edición abierta



Pregunta: ¿Por qué bajó el rendimiento ofensivo?

Modelo debe razonar:

- Reducción de minutos.
- Menos acciones en último tercio.
- Aumento de carga física.



CoT es útil cuando la tarea requiere descomposición lógica o causal:

- **Diagnóstico de rendimiento deportivo** Identificar causas de variaciones en métricas (minutos, carga, acciones).
- **Evaluación táctica** Analizar secuencias de juego paso a paso para inferir decisiones tácticas.
- **Explicaciones causales** Relacionar eventos: fatiga → menor intensidad → menos acciones decisivas.
- **Razonamiento multi-variable** Cuando intervienen estadísticas físicas, técnicas y contextuales.

Efecto: fuerza al modelo a estructurar el proceso antes de la conclusión.



Aunque mejora la coherencia, CoT introduce efectos adversos importantes:

- **Aumento del coste computacional** Generar pasos intermedios incrementa tokens de salida → mayor latencia y coste en sistemas en producción.
- **Ilusión de racionalidad** El modelo puede producir razonamientos estructurados pero incorrectos, dando una falsa sensación de validez.
- **Propagación de errores** Un fallo en un paso intermedio contamina el resto del razonamiento.
- **Mayor exposición a alucinaciones** Cuanto más texto genera el modelo, más oportunidades tiene de introducir información no fundamentada.
- **No aporta verificación factual** CoT mejora forma, no veracidad. No sustituye RAG ni validación externa.

Conclusión: CoT mejora estructura del razonamiento, pero no garantiza corrección.

Errores comunes en Prompt Engineering



La ambigüedad introduce variabilidad no controlada en la salida del modelo.

- No se define con precisión la **tarea objetivo**.
- El modelo debe inferir intención → introduce suposiciones.
- Aumenta la **varianza entre ejecuciones**.
- Complica la evaluación automática y la comparación de resultados.

Impacto en sistemas: salida impredecible → difícil integrar en pipelines. **Ejemplo:** “Analiza el partido” → ¿rendimiento físico, táctica, psicología?



El modelo no detecta “lagunas”; las rellena probabilísticamente.

- Si faltan datos, el modelo usa conocimiento general.
- Aumenta riesgo de **alucinaciones plausibles**.
- Se diluye la frontera entre información real e inferida.
- En deporte: puede inventar estadísticas, roles o situaciones.

Regla operativa: lo que no está en el prompt no forma parte de la realidad del sistema.



Sin formato definido, la salida es texto narrativo no estructurado.

- No se puede validar automáticamente.
- Difícil de almacenar y consultar.
- No integrable en pipelines Big Data.
- Alta variabilidad de estructura entre ejecuciones.

Impacto técnico: se rompe la trazabilidad del sistema. **Solución:** definir siempre esquema de salida (JSON, tabla, claves fijas).



Un prompt multiobjetivo diluye el foco del modelo.

- Clasificar + explicar + evaluar en un solo prompt.
- El modelo reparte capacidad entre subtareas.
- Respuestas más superficiales y menos precisas.
- Mayor probabilidad de incoherencias internas.

Principio de diseño: un prompt → una tarea principal.



La coherencia lingüística no implica veracidad.

- Los LLMs son modelos probabilísticos, no sistemas de verificación.
- Pueden producir respuestas seguras pero incorrectas.
- Mayor riesgo cuando la respuesta parece bien argumentada.
- En deporte: decisiones basadas en datos inventados pueden ser críticas.

Requisito de sistema: validación humana o cruzada con fuentes reales.



Un prompt no evaluado es un componente no controlado del sistema.

- Sin métricas no se puede optimizar ni comparar versiones.
- Los cambios en prompts afectan al comportamiento global del sistema.
- La calidad percibida puede no reflejar estabilidad real.

Resultado: deriva del sistema sin que se detecte.



Un prompt debe evaluarse como cualquier otro componente software.

- Consistencia del formato (JSON válido, claves presentes).
- Tasa de campos vacíos o incorrectos.
- Variabilidad entre ejecuciones con la misma entrada.
- Coherencia con datos reales.
- Estabilidad tras cambios de modelo o temperatura.

Objetivo: minimizar varianza y maximizar fiabilidad operativa.

Post-training de LLMs



- Controla formato y tarea.
- Reduce ambigüedad.
- Mejora consistencia.
- Pero no cambia lo que el modelo sabe ni cómo está alineado.

Por eso existe el post-training.



Un modelo tras el pretraining:

- Ha aprendido a predecir texto.
- Conoce patrones lingüísticos.
- Tiene conocimiento general.

Pero:

- No está optimizado para seguir instrucciones.
- No está alineado con expectativas humanas.
- No está diseñado para ser útil como asistente.



- Conjunto de técnicas que se aplican tras el entrenamiento base.
- Objetivo: modificar el **comportamiento** del modelo.
- No enseña nuevo lenguaje, enseña **cómo responder**.

Idea clave: transforma un generador de texto en un asistente usable.



Ciclo de post-entrenamiento

Esquema conceptual original de la edición abierta



- Que el modelo siga instrucciones.
- Mejorar claridad y estructura de respuestas.
- Controlar tono y estilo.
- Reducir respuestas peligrosas o inapropiadas.
- Enseñar cuándo abstenerse de responder.



- Ajuste con ejemplos humanos correctos.
- El modelo imita respuestas bien estructuradas.
- Introduce estilo conversacional.

Limitación: solo aprende por imitación, no por preferencias.



- Variante de SFT centrada en seguir instrucciones.
- Se entrena con muchas tareas distintas.
- Mejora versatilidad del modelo.

Resultado: menos necesidad de prompts complejos.



- Reinforcement Learning from Human Feedback.
- Humanos comparan respuestas del modelo.
- El modelo aprende qué respuestas son preferidas.

Efecto: mayor naturalidad y alineación con expectativas humanas.



- RLAIIF: feedback generado por modelos IA.
- DPO: optimización directa por preferencias.
- Más simples y escalables que RLHF clásico.



- Adaptación a áreas específicas (medicina, derecho, deporte).
- Mejora precisión terminológica.
- Respuestas más especializadas.

Riesgo: pérdida de generalidad y sesgos de dominio.



- El modelo:
 - Sigue mejor instrucciones.
 - Es más conversacional.
 - Es más prudente.
- Pero:
 - Sigue siendo probabilístico.
 - Sigue alucinando.
 - No tiene acceso a datos externos.



- Prompting = control externo.
- Post-training = comportamiento interno.

Metáfora:

- Post-training educa al modelo.
- Prompting le da instrucciones concretas.



- Alineación imperfecta.
- Alucinaciones persistentes.
- Puede parecer seguro sin ser correcto.
- No sustituye acceso a datos reales.



- Modelos demasiado conservadores.
- Respuestas genéricas y seguras.
- Sesgos heredados del feedback humano.
- Puede priorizar parecer correcto sobre ser útil.



- Un LLM no es una fuente de conocimiento.
- El prompt es una herramienta de ingeniería.
- Las técnicas de prompting mejoran control, no verdad.
- El comportamiento base del modelo viene del post-training.
- Para trabajar con datos reales necesitaremos RAG.



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>